

6장 합성곱 신경망 2 (~6.1.4)

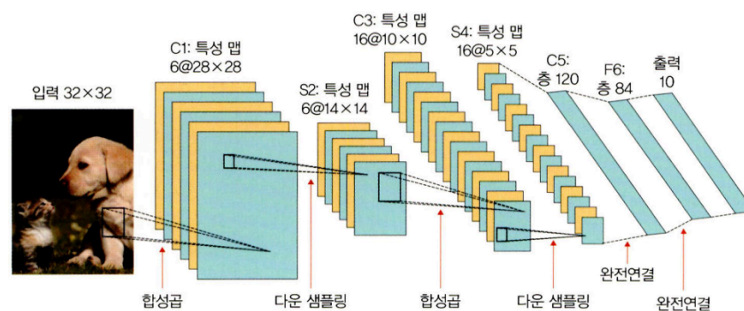
6.1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용한 분류는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것

이미지 분류에 주로 사용되는 합성곱 신경망의 유형

LeNet-5

1995년 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5 발표, 현재 CNN의 초석
LeNet-5는 합성곱과 다운 샘플링을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행



C1에서 5x5 합성곱 연산 후 28x28 크기의 특성 맵 6개를 생성

S2에서 다운 샘플링하여 특성 맵 크기를 14x14로 줄임

C3에서 다시 5x5 합성곱 연산하여 10x10 크기의 특성 맵 16개를 생성

S4에서 다운 샘플링하여 특성 맵 크기를 5x5로 줄임

C5에서 5x5 합성곱 연산하여 1x1 크기의 특성 맵 120개를 생성

F6에서 완전연결층으로 C5의 결과를 유닛 84개에 연결

(C : 합성곱층, S : 풀링층, F : 완전 연결층)

▼ 왜 이런 과정을 반복할까?

처음에는 이미지의 단순한 특징 (선, 모서리 등)을 찾고

점점 더 복잡하고 중요한 특징을 찾으면서

이미지의 크기는 점점 줄여서 마지막에는 그 특징을 이용해 이미지를 분류하기 위해서

입력 이미지 → 선, 모서리 같은 특징 찾기 → 중요한 특징은 남기고 크기 줄이기 → 조금

더 복잡한 특징 찾기 → 다시 크기 줄이기 → 이미지를 구별하는 고수준 특징 찾기 → 완

전 연결

C1에서 합성곱 필터들이 이미지의 작은 영역을 보면서 기초적인 특징을 찾아 특성 맵 6개를 만들

각 특성 맵은 이 이미지에서 특정 특징이 어디에 강하게 나타나는지를 나타냄

S2에서는 만든 특성 맵의 크기를 줄임

28x28짜리 특성 맵을 계속 큰 크기로 가지고 있으면 계산량도 많고 위치가 조금 달라지는 것에 너무 민감할 수 있어 다운 샘플링을 함

정확히 특징이 어디에 있는지 기억하기보다는 이 근처에 이런 특징이 있다는 정도로 정보를 압축

→ 풀링 = 중요한 특징을 어느 정도 유지하면서 공간 크기를 줄이는 과정

C3에서 다시 합성곱을 진행

C1에서는 단순한 특징을 찾았지만 이미지를 분류하려면 단순한 특징만으로는 부족함
여러 선과 모서리가 어떤 형태로 조합되어 있는지 봐야함

그래서 다시 합성곱을 진행함

= 앞에서 찾은 특징들을 이용해서 더 복잡한 특징을 찾는 과정

S4에서 찾은 더 복잡한 특징들을 다시 압축함

점점 세부적인 위치 정보는 줄어 들고 중요한 특징에 집중하게 됨

S4에서 특성 맵이 5x5가 되어서 C5에서 5x5 필터를 적용하면 1x1가 출력됨

즉 특성 맵 하나가 하나의 값으로 압축됨

= 앞의 여러 합성곱과 풀링을 거치면서 이미지의 중요한 특징들을 계속 압축했고, C5에서는 그 특징들을 120개의 숫자로 표현한 것

F6에서 연결함

이제 특징을 찾는 것에서 분류하는 것으로 역할이 바뀜

C5에서 120개의 값들을 이용해서 판단해야 함

완전 연결층에서 C5의 특징들을 서로 조합해서 최종적인 판단을 함

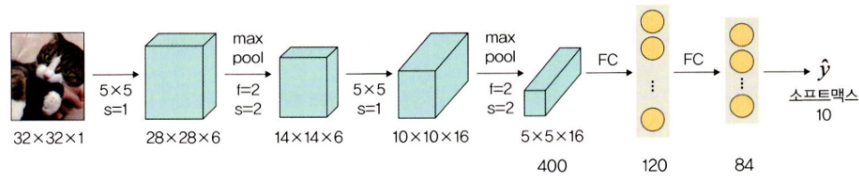
합성곱층 = 특징을 추출하는 역할

풀링층 = 특징을 유지하면서 크기를 줄이는 역할

완전연결층 = 추출된 특징을 종합해서 분류하는 역할

이미지에서 단순한 특징을 찾음 → 크기를 줄임 → 그 특징들을 이용해 더 복잡한 특징을 찾음 → 다시 줄임 → 마지막에 특징들을 종합해서 이미지를 분류

합성곱과 풀링을 반복하는 이유 : 특징을 점점 더 추상적이고 복잡하게 만들어가기 위해서



예제에서 구현할 LeNet-5 신경망

32x32 크기의 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

LeNet-5 예제 신경망 상세

합성곱층과 완전연결층의 활성화 함수로는 ReLU, 마지막 완전연결층에는 softmax를 사용할 예정

이미지 데이터 전처리 및 데이터셋 구성

```
# 이미지 데이터셋 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([ # 이미지를 변형할 수 있는 방식들의 묶음
                transforms.RandomHorizontalFlip(), #기본 확률값 0.5
                transforms.ToTensor(), #2d의 범위와 배열의 차원을 바꾸어 파이토치에서 사용가능하게 함
                transforms.Normalize(mean, std) #imageNet 데이터의 각 채널별 평균과 표준편차에 맞게 정규화
            ]),
            'validation': transforms.Compose([
                transforms.Resize(256), #같은 비율 256으로 맞추고 원래 비율은 유지
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    #테스트를 호출할 수 있도록 하는 메서드
    def __call__(self, img, phase):
        return self.data_transform[phase](img)

# 이미지 데이터를 불러온 후 훈련, 검증, 테스트용 분리
cat_directory = r'content/sample_data/chap09/dogs-vs-cats/Cat/'
dog_directory = r'content/sample_data/chap09/dogs-vs-cats/Dog/'

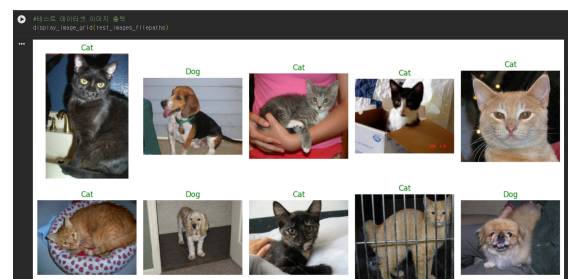
cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
                                os.listdir(dog_directory)])

images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
#다 이상 데이터를 찾을 수 없을 때까지 모든 이미지 데이터를 읽어옴
correct_images_filepaths = [f for f in images_filepaths if cv2.imread(f) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] #훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:10] #검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[10:] #테스트용 10개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

400 92 10
```

```
# 테스트 데이터셋 이미지 확인 함수
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # 이미지의 색상 변경
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2] # 이미지 전체 경로 정규화, 분할
        predicted_label = predicted_labels[i] if predicted_labels else true_label
        color = "green" if true_label == predicted_label else "red"
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(predicted_label, color=color)
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```



실습에서 개와 고양이 이미지 데이터를 이용하여 이미지 분류 모델을 학습하기 위한 데이터 전처리 과정 수행

ImageTransform 클래스를 정의하여 학습 데이터와 검증 데이터에 서로 다른 전처리 방법 적용

학습 데이터에는 RandomResizedCrop과 RandomHorizontalFlip을 적용하여 이미지의 크기와 방향을 다양하게 변형하는 데이터 증강 수행

이후 ToTensor를 통해 이미지를 PyTorch Tensor로 변환하여 Normalize를 이용하여 ImageNet의 평균과 표준편차를 기준으로 정규화함

검증 데이터에는 랜덤한 변형을 적용하지 않음

Resize와 CenterCrop을 이용하여 일정한 방식으로 이미지 크기를 조정한 후 Tensor 변환과 정규화 수행

이후 고양이와 강아지 이미지가 저장된 디렉터리에서 이미지 파일의 경로를 가져와 하나의 리스트로 통합

OpenCV의 imread()를 이용하여 실제로 읽을 수 있는 정상적인 이미지만 선별

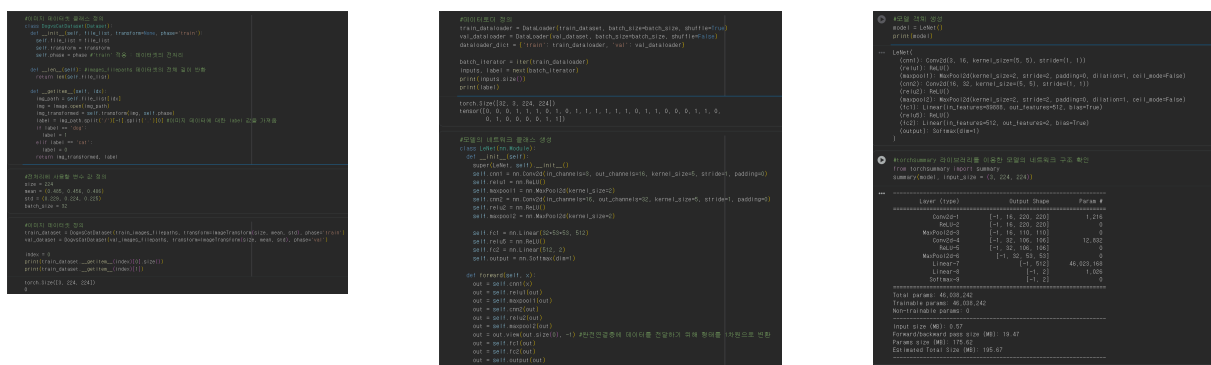
전체 데이터를 학습, 검증, 테스트 데이터로 분리

학습 데이터는 400장, 테스트 데이터는 10장으로 구성하고 나머지는 검증 데이터로 사용

테스트 이미지들을 확인하기 위해 `display_image_grid()` 함수를 정의

OpenCV로 이미지를 읽은 후 BGR 형식을 RGB 형식으로 변환하여 Matplotlib을 이용하여 격자 형태로 출력

이미지 경로에서 실제 클래스 정보를 추출하여 이미지의 정답 라벨을 확인할 수 있도록 구성함



DogvsCatDataset 클래스를 정의하여 이미지 파일을 불러오고, 앞에서 설정한 전처리를 적용한 후 고양이는 0, 강아지는 1로 라벨링

`Dataloader`를 이용해 학습과 검증 데이터를 배치 크기 32로 구성

224×224 크기의 RGB 이미지를 입력으로 받는 LeNet 기반 CNN 모델을 구성

두 개의 합성곱층과 ReLU, 최대 풀링을 이용해 이미지의 특징을 추출하고 추출된 특징을 완전연결층에 전달하여 고양이와 강아지 두 클래스로 분류하도록 설계

torchsummary를 이용해 모델의 각 계층과 전체 네트워크 구조를 확인

```
#학습 가능한 파라미터 수 확인
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')

#옵티마이저와 손실 함수 정의
optimizer = optim.Adam(model.parameters()) #lr=0.001, amsgrad=False
criterion = nn.CrossEntropyLoss()

#모델의 파라미터와 손실 함수를 CPU에 할당
model = model.to(device)
criterion = criterion.to(device)
```

```
#모델 학습 함수 정의
def train_model(model, dataloader_dict, criterion, optimizer, num_epochs):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}:'.format(epoch + 1, num_epochs))
        print('-' * 20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                if phase == 'train':
                    loss.backward()
                    optimizer.step()

                epoch_loss += loss.item() * inputs.size(0)
                epoch_corrects += torch.sum(preds == labels.data)

            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = model.state_dict()

        time_elapsed = time.time() - since
        print('Training complete in {:.0f}m {:.0f}s'.format(
            time_elapsed // 60, time_elapsed % 60))

    print('Best val Acc: {:.4f}'.format(best_acc))

    return model
```

```
#모델 학습
import time

num_epochs = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epochs)
Epoch 4/10

100% |#####| 13/13 [00:25<00:00, 1.72s/it]
train Loss: 0.6782 Acc: 0.5925
100% |#####| 3/3 [00:02<00:00, 1.17s/it]
val Loss: 0.7079 Acc: 0.4665
Epoch 5/10

100% |#####| 13/13 [00:25<00:00, 1.69s/it]
train Loss: 0.6722 Acc: 0.5875
100% |#####| 3/3 [00:02<00:00, 1.17s/it]
val Loss: 0.7081 Acc: 0.5000
Epoch 6/10

100% |#####| 13/13 [00:25<00:00, 1.67s/it]
train Loss: 0.6707 Acc: 0.5930
100% |#####| 3/3 [00:02<00:00, 1.15s/it]
val Loss: 0.7019 Acc: 0.4891
Epoch 7/10

100% |#####| 13/13 [00:25<00:00, 1.89s/it]
train Loss: 0.6551 Acc: 0.6325
100% |#####| 3/3 [00:02<00:00, 1.18s/it]
val Loss: 0.7022 Acc: 0.4783
Epoch 8/10

100% |#####| 13/13 [00:25<00:00, 1.78s/it]
train Loss: 0.6527 Acc: 0.6075
100% |#####| 3/3 [00:02<00:00, 1.17s/it]
val Loss: 0.6983 Acc: 0.5108
Epoch 9/10

100% |#####| 13/13 [00:25<00:00, 1.72s/it]
train Loss: 0.6550 Acc: 0.6250
100% |#####| 3/3 [00:02<00:00, 1.17s/it]
val Loss: 0.6915 Acc: 0.5217
Epoch 10/10

100% |#####| 13/13 [00:25<00:00, 1.69s/it]
```

모델의 학습 가능한 파라미터 수를 확인하고 SGD 옵티마이저와 CrossEntropyLoss 손실 함수를 설정

train_model() 함수를 통해 10번의 epoch 동안 학습과 검증을 반복하도록 구성

학습 단계에서는 예측값과 실제 라벨의 차이를 이용해 손실을 계산하고 역전파와 옵티마이저를 통해 모델의 파라미터를 업데이트함

검증 단계에서는 파라미터를 업데이트하지 않고 loss와 정확도를 확인

각 epoch의 검증 정확도를 비교하여 가장 높은 성능을 기록하고, 전체 학습이 끝난 후 최고 검증 정확도와 학습 시간을 출력

```
#테스트 결과를 저장할 함수 정의
def test_model(model, dataloader_dict):
    since = time.time()
    best_acc = 0.0

    for phase in ['train', 'val']:
        model.eval()

        epoch_loss = 0.0
        epoch_corrects = 0

        for inputs, labels in tqdm(dataloader_dict[phase]):
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad():
                outputs = model(inputs)
                _, preds = torch.max(outputs, 1)
                loss = criterion(outputs, labels)

            epoch_loss += loss.item() * inputs.size(0)
            epoch_corrects += torch.sum(preds == labels.data)

        epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
        epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

        print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    time_elapsed = time.time() - since
    print('Testing complete in {:.0f}m {:.0f}s'.format(
        time_elapsed // 60, time_elapsed % 60))

    print('Best test Acc: {:.4f}'.format(best_acc))

    return model
```

```
#테스트 결과, 이미지와 함께 출력
res = test_model(model, dataloader_dict)

res.head(10)
```

	id	label
0	109	0.476350
1	145	0.409545
2	15	0.622046
3	162	0.522682
4	167	0.502213
5	200	0.459493
6	210	0.631383
7	211	0.618819
8	213	0.454131
9	224	0.577926

다음 단계: [res 변수로 코드 생성](#) [New interactive sheet](#)

```
#테스트 데이터의 이미지를 출력하고, 예측한 학습 결과
class_ = classes = [0: 'cat', 1: 'dog']
def display_image_grid(images, filepaths, predicted_label=(), cols=5):
    rows = len(images) // cols
    figure, ax = plt.subplots(rows, cols, figsize=(12, 8))
    for i, image_filepath in enumerate(filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        a = random.choice(res['id'].values)
        label = res.loc(res['id'] == a, 'label').values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()

plt.tight_layout()
plt.show()
```



학습이 완료된 모델을 이용하여 10개의 테스트 이미지에 대한 예측을 수행

테스트 이미지에 검증용 전처리를 적용한 후 모델에 입력하고 softmax를 통해 강아지일 확률을 계산

예측 결과는 이미지의 ID와 함께 dataframe으로 저장

display_image_grid() 함수를 이용하여 테스트 이미지를 격자 형태로 출력하고, 모델이 예측한 고양이, 강아지 라벨을 이미지 위에 표시하여 예측 결과를 시각적으로 확인함

코드에서 네트워크의 각 부분들을 통과할때마다 입력과 출력의 형태가 바뀜
그 계산은 다음 수식과 같음

Conv2d 계층에서의 출력 크기 구하는 공식

출력 크기 = $(W - F + 2P) / S + 1$

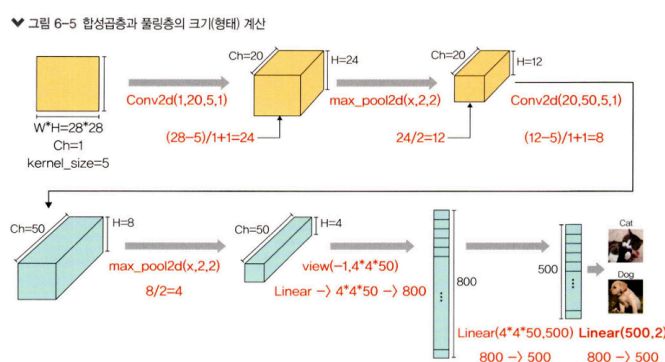
- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하는 공식

출력 크기 = IF / F

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

계산 수식을 이용한 각각의 계층에 대한 결과



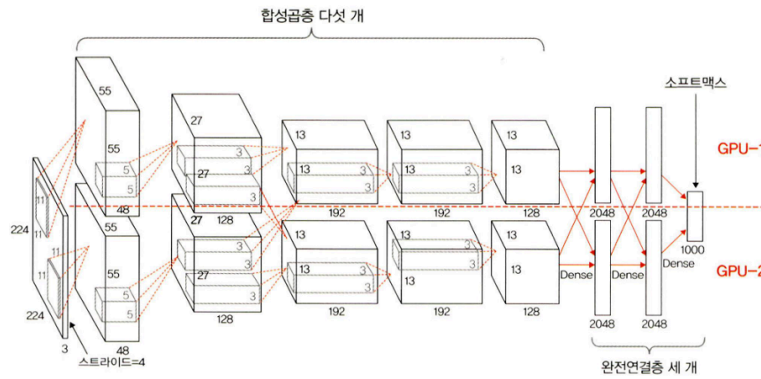
AlexNet

ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회에서 우승한 CNN 구조

CNN은 3차원 구조를 가짐

이미지 크기를 나타내는 너비와 높이 뿐만 아니라 깊이를 갖고 있음

색상이 많은 이미지는 RGB 성분 3개를 갖기 때문에 시작이 3이지만 합성곱을 거치면서 특성 맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라짐



AlexNet 구조

AlexNet은 합성곱층 총 5개와 완전 연결층 3개로 구성

합성곱층에서 사용된 활성화 함수는 ReLU

맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수 사용

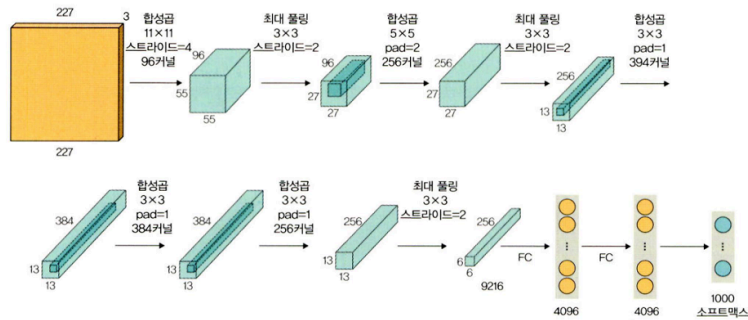
GPU 두개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않음

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

AlexNet 구조 상세

AlexNet의 첫번째 합성곱층 커널의 크기는 11x11x3으로 stride 4를 적용하면 특성 맵을 96개 생성하여 55x55x96의 출력을 가짐

첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관 없는 정보를 추출하기 위한 커널이 학습되고 GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨



AlexNet 예제 네트워크

```
#DogvsCat 데이터셋 정의
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = image.open(img_path)
        img_transformed = self.transform(img, self.phase)

        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0

        return img_transformed, label

#변조된 데이터셋 정의
size = 256
mean = (0.485, 0.486, 0.486)
std = (0.229, 0.224, 0.229)
batch_size = 32

#훈련, 검증, 테스트 데이터셋 정의
train_dataset = DogvsCatDataset(train_image_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset = DogvsCatDataset(val_image_filepaths, transform=ImageTransform(size, mean, std), phase='val')
test_dataset = DogvsCatDataset(test_image_filepaths, transform=ImageTransform(size, mean, std), phase='test')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])

torch.Size([3, 256, 256])
0
```

```
#데이터셋을 메모리로 불러옴
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

torch.Size([32, 3, 256, 256])
tensor([1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1,
        1, 1, 1, 1, 1, 0, 0, 0])
```

LeNet-5와 마찬가지로 DogvsCatDataset 클래스를 이용해 이미지를 불러오고 전처리한 뒤 고양이는 0, 강아지는 1로 라벨링

AlexNet의 입력 크기에 맞추기 위해 이미지 크기를 256x256으로 설정하고 ImageNet의 평균과 표준편차를 이용해 정규화

학습, 검증, 테스트용 Dataset을 각각 생성한 후 DataLoader를 이용하여 배치 크기 32로 데이터를 묶음

학습 데이터는 shuffle=True로 설정하여 데이터 순서를 섞고, 검증과 테스트 데이터는 shuffle=False로 설정

학습 데이터에서 하나의 배치를 가져와 입력 이미지의 크기와 라벨이 정상적으로 구성되었는지 확인

```

class AlexNet(nn.Module):
    def __init__(self) > None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 128, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(128, 256, kernel_size=5, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(256, 256, kernel_size=5, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2)
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256*6*6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 100)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

[illegible]

```

11  from torchsummary import Summary
12  summary(model, input_size = (3, 256, 256))
13
14  ...
15
16  Layer (type)                   Output Shape          Param #
17  ---
18  Conv2d-1                       [-1, 64, 16, 16]     23,386
19  ReLU-2                          [-1, 64, 16, 16]
20  Conv2d-3                       [-1, 64, 83, 63]     0
21  ReLU-4                          [-1, 64, 31, 31]
22  MaxPool2d-5                    [-1, 162, 31, 31]    307,382
23  Conv2d-6                       [-1, 192, 31, 31]     0
24  ReLU-7                          [-1, 192, 31, 31]
25  MaxPool2d-8                    [-1, 192, 15, 15]     0
26  Conv2d-9                       [-1, 128, 15, 15]    663,886
27  ReLU-10                        [-1, 128, 15, 15]
28  Conv2d-11                      [-1, 384, 15, 15]     0
29  ReLU-12                        [-1, 384, 15, 15]
30  Conv2d-13                      [-1, 192, 15, 15]    884,192
31  ReLU-14                        [-1, 192, 15, 15]
32  Conv2d-15                      [-1, 256, 15, 15]     0
33  ReLU-16                        [-1, 256, 15, 15]    590,080
34  Conv2d-17                      [-1, 256, 7, 7]
35  ReLU-18                        [-1, 256, 7, 7]
36  MaxPool2d-19                  [-1, 256, 6, 6]
37  Adapt LeakyReLU-20             [-1, 256, 6, 6]
38  Dropout-19                    [-1, 256]
39  Linear-16                     [-1, 4096]
40  Linear-17                     [-1, 4096]
41  Dropout-18                    [-1, 4096]
42  Linear-19                     [-1, 512]
43  Linear-20                     [-1, 512]
44  Linear-21                     [-1, 2]
45  1,026
46
47  Total params: 42,321,218
48  Trainable params: 42,321,218
49  Non-trainable params: 0
50
51  Input size (MB): 0.75
52  Forward/backward pass size (MB): 10.90
53  Param size (MB): 161.44
54  Estimated total size (MB): 173.10

```

AlexNet 구조를 기반으로 개와 고양이를 분류하는 CNN 모델을 구성

Features 부분에서 여러 개의 합성곱층 + ReLU + MaxPooling을 반복하여 이미지의 특징을 추출

앞쪽에서는 비교적 큰 커널을 사용하여 기본적인 특징을 추출하고 뒤로 갈수록 여러 합성곱 층을 거치면서 더 복잡하고 추상적인 특징을 추출

이후 AdaptiveAvgPool2d를 이용해 특징 맵의 크기를 6x6으로 일정하게 맞추고, 이를 1차원으로 펼쳐 classifier에 전달

완전연결층에서는 dropout를 사용하여 과적합을 줄임

최종적으로 2개의 출력값을 만들어 고양이와 강아지를 분류하도록 구성

SGD 옵티마이저와 CrossEntropyLoss를 사용하여 모델을 학습할 수 있도록 설정
torchsummary를 통해 256x256 입력 이미지에 대한 AlexNet의 전체 네트워크 구조를
확인

```

import time

def train_epoch(epoch, dataloader_dict, criterion, optimizer, num_epochs):
    time.time()
    best_acc = 0.0

    for epoch in range(1, num_epochs):
        print('Epoch {}/{}: format epoch + 1, num_epochs')
        print('-' * 40)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                optimizer.zero_grad()

            with torch.set_grad_enabled(phase == 'train'):
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()

            epoch_loss += loss.item() * inputs.size(0)
            epoch_corrects += torch.sum(outputs == labels.data)

            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

            print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

    print('train: loss: {} | acc: {}' .format(
        print('Training complete in: {:.0f} | {:.0f}' .format(
            time.time() - start, time.time() - end))
    ))

```

```

# GPU 메모리
max_epochs = 10
model = train_model(model, dataloader_dict, criterion, optimizer, max_epochs)

# 100%
13/13 [00:45-00:00, 3.12s/it]
train Loss : 0.690 Acc : 0.4975

# 100%
3/3 [00:03-00:00, 1.21s/it]
val Loss : 0.6941 Acc : 0.4591
Epoch 4/10

# 100%
13/13 [00:44-00:00, 3.05s/it]
train Loss : 0.6934 Acc : 0.4975

# 100%
3/3 [00:04-00:00, 1.40s/it]
val Loss : 0.6937 Acc : 0.4691
Epoch 5/10

# 100%
13/13 [00:44-00:00, 2.95s/it]
train Loss : 0.6905 Acc : 0.4975

# 100%
3/3 [00:04-00:00, 1.46s/it]
val Loss : 0.6935 Acc : 0.4691
Epoch 6/10

# 100%
13/13 [00:44-00:00, 3.07s/it]
train Loss : 0.6907 Acc : 0.4975

# 100%
3/3 [00:03-00:00, 1.22s/it]
val Loss : 0.6923 Acc : 0.4591
Epoch 7/10

# 100%
13/13 [00:45-00:00, 3.21s/it]
train Loss : 0.6829 Acc : 0.4975

# 100%
3/3 [00:03-00:00, 1.20s/it]
val Loss : 0.6888 Acc : 0.4691
Epoch 8/10

```

[illegible]

train_model() 함수를 이용해 AlexNet을 10 epoch 동안 학습시킴

학습 과정에서 CrossEntropyLoss로 손실을 계산하고, SGD를 이용하여 모델의 가중치를

업데이트

각 epoch마다 학습 및 검증 데이터의 Loss와 Accuracy를 확인

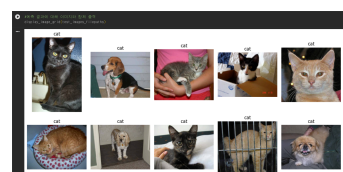
학습된 AlexNet을 이용해 테스트 이미지의 고양이와 강아지를 분류

테스트 이미지에 검증용 전처리를 한 후 모델의 출력값에 softmax를 적용하여 강아지 클래스의 예측 확률을 구함

```
#데이터 프레임 결과 확인
res.head(10)
```

	id	label
0	145	0.497924
1	211	0.498681
2	162	0.498175
3	200	0.500008
4	210	0.499034
5	224	0.498128
6	213	0.498035
7	109	0.499567
8	15	0.497784
9	167	0.497682

```
#예측 결과를 시각적으로 표현하기 위한 함수 정의
from matplotlib import pyplot as plt
class_ = classes = (0, 'cat', 1, 'dog')
def display_image_grid(image_filenames, predicted_labels, cols):
    rows = len(image_filenames) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 8))
    for i, image_filename in enumerate(image_filenames):
        image = cv2.imread(image_filename)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        x = random.choice(class_)
        label = res.loc[res['id'] == i, 'label'].values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```



AlexNet의 예측 결과 확인

예측 확률을 기준으로 0.5보다 크면 dog, 작거나 같으면 cat으로 분류
테스트 이미지에 예측된 클래스를 표시하여 결과를 시각적으로 확인함

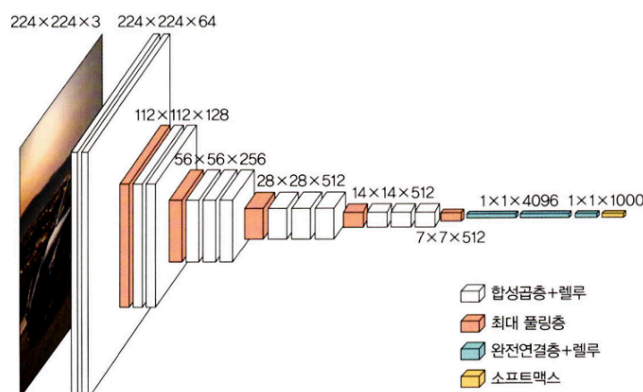
데이터셋이 적어 예측 결과가 좋지 않음

VGGNET

합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생

네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 네트워크

깊이의 영향만 확인하고자 합성곱층에서 사용하는 필터, 커널의 크기를 가장 작은 3x3으로 고정



VGG16 구조

VGGNet 중에서 가장 간단한 VGG11을 파이토치로 구현

[illegible]

얕은 복사(shallow copy) 깊은 복사(deep copy)

A B A B

메모리 공간

메모리 공간 1개 메모리 공간 2개

11

```
#0001 사전 훈련된 모델 사용
import torchvision.models as models
pretrained_model = models.vgg11(pretrained=True)
print(pretrained_model)

#데이터 데이터 전처리
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(224, 224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

#ImageFolder를 이용하여 데이터셋 불러오기
train_path = f'/content/sample_data/chap06/vgg11/train'
test_path = f'/content/sample_data/chap06/vgg11/test'

train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
)

test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)

print(len(train_dataset), print(len(test_dataset))

529
12
None, None)
```

```
#훈련과 검증 데이터 분할
#데이터셋을 나누기
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=100, shuffle=True, num_workers=4)

#검증 데이터 전처리
valid_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=100, shuffle=True, num_workers=4)

#모델을 검증하는 함수
def validate(model, data_loader, criterion, device):
    model.eval()
    for batch_idx, (data, target) in enumerate(data_loader):
        data, target = data.to(device), target.to(device)
        output = model(data)
        loss = criterion(output, target)
        total_loss += loss.item()
    return total_loss / (len(data_loader) + 1)

#모델을 검증하는 함수
def validate(model, data_loader, criterion, device):
    model.eval()
    for batch_idx, (data, target) in enumerate(data_loader):
        data, target = data.to(device), target.to(device)
        output = model(data)
        loss = criterion(output, target)
        total_loss += loss.item()
    return total_loss / (len(data_loader) + 1)
```

```
#모델 학습 함수
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred, _ = model(x)
        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)
        epoch_loss += loss.item()
        epoch_acc += acc.item()

    return epoch_loss / len(iterator), epoch_acc / len(iterator)

#모델 성능 측정 함수
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()

    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

ImageNet으로 사전 학습된 VGG11 모델을 불러오고, 훈련/검증/테스트 이미지에 필요한 전처리 적용

ImageFolder로 데이터를 불러와 훈련 데이터와 검증 데이터로 나누고 DataLoader를 이용해 배치 단위로 모델에 입력할 수 있도록 준비

Adam 옵티마이저와 CrossEntropyLoss를 설정하고 train(), evaluate() 함수를 통해 모델을 학습하고 성능을 평가

검증 데이터에는 copy.deepcopy()를 사용하여 원본 데이터셋에 영향을 주지 않으면서 테스트 데이터와 동일한 전처리를 적용

```
#학습 시간 측정 함수
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs

#모델 학습
import time
EPOCHS = 5
best_valid_loss = float('inf')

for epoch in range(EPOCHS):
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_loader, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_loader, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), f'/content/sample_data/chap06/VGG-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'Train Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'Valid Loss: {valid_loss:.3f} | Valid Acc: {valid_acc*100:.2f}%')

Epoch: 01 | Epoch Time : 11m 43s
Train Loss: 0.710 | Train Acc: 46.12%
Valid Loss: 0.695 | Valid Acc: 41.51%
Epoch: 02 | Epoch Time : 11m 42s
Train Loss: 0.691 | Train Acc: 55.08%
Valid Loss: 0.696 | Valid Acc: 43.40%
```

```
#테스트 데이터셋을 이용한 모델 성능 측정
model.load_state_dict(torch.load('/content/sample_data/chap06/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_loader, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')

Test Loss: 0.690 | Test Acc: 50.00%

#테스트 데이터셋을 이용한 모델의 예측 확인 함수
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim=-1)
            top_pred = y_prob.argmax(1, keepdim=True)
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())

    images = torch.cat(images, dim=0)
    labels = torch.cat(labels, dim=0)
    probs = torch.cat(probs, dim=0)
    return images, labels, probs

#예측 중에서 정확하게 예측한 것을 추출
images, labels, probs = get_predictions(model, test_loader)
pred_labels = torch.argmax(probs, 1)
corrects = torch.eq(labels, pred_labels)
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects):
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)
```

Epochs=5로 설정하여 모델을 총 5번 학습하고 각 epoch마다 훈련 데이터로 학습한 후 검증 데이터의 loss와 accuracy를 확인

이전보다 valid_loss가 낮아지면 해당 모델의 가중치를 VGG-model.pt로 저장하여 가장 성능이 좋은 모델을 보존

학습이 끝난 후 저장된 최적 모델을 다시 불러와 테스트 데이터셋의 Loss와 accuracy를 측정

get_predictions() 함수를 통해 테스트 이미지에 대한 예측 확률을 계산

실제 label과 예측 label을 비교하여 정확하게 예측한 이미지를 추출

추출된 정확한 예측 결과는 모델의 예측 확률이 높은 순서로 정렬하여 모델이 확신을 가지고 올바르게 분류한 이미지를 확인할 수 있도록 함

train accuracy는 55.08%인데 Test accuracy는 50.00%에 머물러 있어, 현재 결과 만으로는 모델이 데이터의 특징을 충분히 학습했다고 보기 어려움

```
#이미지 출력에 위한 전처리
def normalize_image(image):
    image_min = image.min()
    image_max = image.max()
    image.clamp_(min=image_min, max=image_max)
    image.add_(-image_min).div_(image_max-image_min+1e-5)
    return image

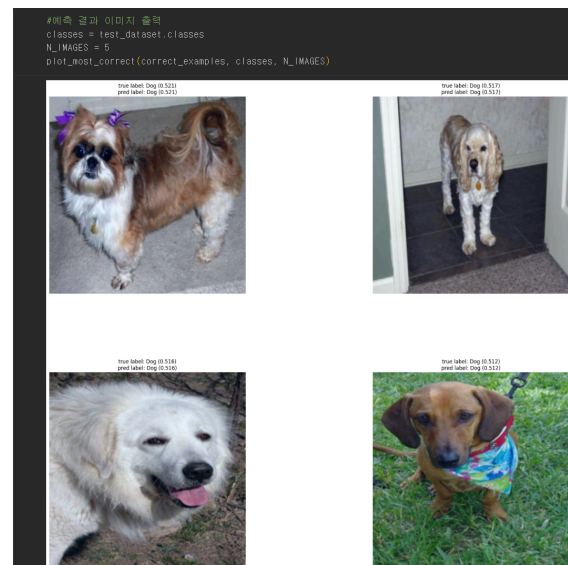
#모델이 정확하게 예측한 이미지 출력 함수
from matplotlib import pyplot as plt
def plot_most_correct(correct, classes, n_images, normalize=True):
    rows = int(np.sort(n_images))
    cols = int(np.sort(n_images))
    fig = plt.figure(figsize=(25, 20))

    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1)
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0)
        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim=0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

        if normalize:
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n' #
                    f'pred label: {correct_class} ({correct_prob:.3f})')
        ax.axis('off')

    fig.subplots_adjust(hspace=0.4)
```



normalize_image() 함수를 활용하여 정규화된 이미지의 픽셀값을 0~1 범위로 조정해 이미지가 정상적으로 보이도록 함

permute()를 사용해 이미지의 차원 순서를 Matplotlib에 맞게 변경하고 실제 클래스와 예측 클래스 및 예측 확률을 이미지 위에 표시

N_IMAGES = 5로 설정하여 모델이 정확하게 예측한 이미지 5개를 시각적으로 확인

결과의 정확도가 높지 않음

데이터셋을 늘려 준다면 정확도에 대한 성능이 높아질 것

GoogLeNet

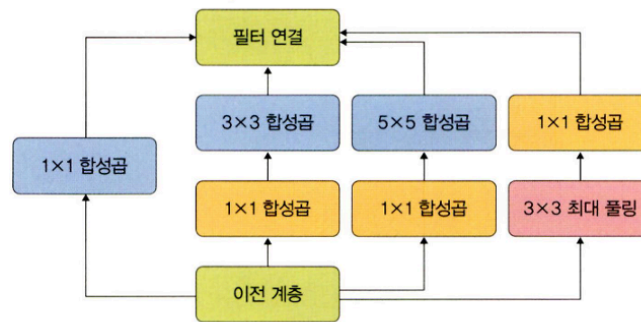
주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망

그런 신경망을 위해 인셉션 모듈 추가함

인셉션 모듈은 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩

을 추가해야 함

이를 해결하기 위한 방법은 희소 연결 (sparse connectivity)



CNN은 합성곱, 풀링, 완전연결층들이 서로 밀집하게 연결되어있음

대신 관련성이 높은 노드끼리만 연결하는 방법 : 희소 연결

연산량이 적어지며 과적합도 해결 가능

딥러닝을 이용하면 대용량 데이터를 학습해야 함

심층 신경망의 아키텍처에서 계층이 넓고 깊으면 인식률이 좋아지지만 과적합이나 기울기 소멸 문제 등이 발생함

특히 합성곱 신경망에서 이러한 문제들이 자주 발생

GoogLeNet은 이러한 문제를 해결할 수 있음